

Phase 1: Continued Pre-training on Medical Literature (GPU)

Train Qwen2.5-7B on 14 medical books using LoRA on RTX 5090

- **Duration:** 12-14 GPU hours (RTX 5090)
- **Memory:** ~20-24 GB VRAM
- **Output:** Medical-grounded Qwen model ready for Phase 2

1. Setup and Environment

```
In [1]: import os
os.environ['TOKENIZERS_PARALLELISM'] = 'false'
os.environ['CUDA_DEVICE_ORDER'] = 'PCI_BUS_ID'

import torch
import json
from pathlib import Path
from datetime import datetime
import warnings
warnings.filterwarnings('ignore')

print("="*80)
print("PHASE 1: CONTINUED PRE-TRAINING (GPU)")
print("="*80)
print(f"Timestamp: {datetime.now().isoformat()}")
print(f"Device: {torch.device('cuda' if torch.cuda.is_available() else 'cpu')}")
if torch.cuda.is_available():
    print(f"GPU: {torch.cuda.get_device_name(0)}")
    print(f"VRAM: {torch.cuda.get_device_properties(0).total_memory / 1e9:.1f} GB")
    print(f"CUDA Version: {torch.version.cuda}")
print("="*80)
print()
```

```
=====
PHASE 1: CONTINUED PRE-TRAINING (GPU)
=====
```

```
Timestamp: 2026-05-13T17:30:21.007633
```

```
Device: cuda
```

```
GPU: NVIDIA GeForce RTX 5090
```

```
VRAM: 34.2 GB
```

```
CUDA Version: 12.8
=====
```

2. Verify Data

```
In [2]: DATA_FILE = 'full_medical_data.txt'
OUTPUT_DIR = 'qwen_medical_pretrained_gpu'
LORA_OUTPUT = 'qwen_medical_lora_gpu'
CACHE_DIR = '.cache'

Path(OUTPUT_DIR).mkdir(exist_ok=True, parents=True)
Path(LORA_OUTPUT).mkdir(exist_ok=True, parents=True)
Path(CACHE_DIR).mkdir(exist_ok=True, parents=True)

print("[1/6] Verifying data file...")

if not os.path.exists(DATA_FILE):
    print(f"ERROR: Data file not found: {DATA_FILE}")
    print(f"Run: extract_pdf_data.py")
else:
    data_size_mb = os.path.getsize(DATA_FILE) / (1024 * 1024)
    with open(DATA_FILE, 'r', encoding='utf-8') as f:
        text = f.read()
        word_count = len(text.split())

    print(f" ✓ Data file verified")
    print(f" Path: {DATA_FILE}")
    print(f" Size: {data_size_mb:.2f} MB")
    print(f" Words: {word_count:,}")
    print()
```

```
[1/6] Verifying data file...
 ✓ Data file verified
Path: full_medical_data.txt
Size: 5.76 MB
Words: 899,042
```

3. Load Tokenizer

```
In [3]: from transformers import AutoTokenizer

MODEL_NAME = "Qwen/Qwen2.5-7B"

print("[2/6] Loading tokenizer...")
tokenizer = AutoTokenizer.from_pretrained(
    MODEL_NAME,
    trust_remote_code=True,
    cache_dir=CACHE_DIR
)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token

print(f" ✓ Tokenizer loaded")
print(f" Model: {MODEL_NAME}")
print(f" Vocab size: {len(tokenizer)}")
print()
```

```
[2/6] Loading tokenizer...
      ✓ Tokenizer loaded
      Model: Qwen/Qwen2.5-7B
      Vocab size: 151665
```

4. Load Base Model

```
In [7]: from transformers import AutoTokenizer, AutoModelForCausalLM

# Load from local path
LOCAL_MODEL_PATH = 'Qwen2.5-7B' # Adjust path as needed

print("[3/6] Loading base model from local...")

model = AutoModelForCausalLM.from_pretrained(
    LOCAL_MODEL_PATH,
    torch_dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)

print(f" ✓ Model loaded from: {LOCAL_MODEL_PATH}")
print(f" Parameters: {sum(p.numel() for p in model.parameters()) / 1e9:.1f}B")
print(f" Device: {next(model.parameters()).device}")
print()
```

```
[3/6] Loading base model from local...
```

```
Loading weights: 100%|██████████  
██████████| 339/339 [00:04<00:00, 74.97it/s]  
✓ Model loaded from: Qwen2.5-7B  
Parameters: 7.6B  
Device: cuda:0
```

5. Configure LoRA

```
In [8]: from peft import get_peft_model, LoraConfig, TaskType

print("[4/6] Configuring LoRA...")

lora_config = LoraConfig(
    task_type=TaskType.CAUSAL_LM,
    r=16,                                # Rank
    lora_alpha=32,                        # Alpha (32/16 = 2.0)
    lora_dropout=0.05,
    target_modules=["q_proj", "v_proj"],
    bias="none",
)

model = get_peft_model(model, lora_config)
trainable_params = sum(p.numel() for p in model.parameters() if p.requires_grad)
all_params = sum(p.numel() for p in model.parameters())
```

```

print(f" ✓ LoRA configured")
print(f"  LoRA rank: {lora_config.r}")
print(f"  LoRA alpha: {lora_config.lora_alpha}")
print(f"  Trainable params: {trainable_params / 1e6:.1f}M ({100 * trainable_params / 1e9:.1f}%)")
print(f"  Total params: {all_params / 1e9:.1f}B")
print()

model.print_trainable_parameters()

```

[4/6] Configuring LoRA...

```

✓ LoRA configured
LoRA rank: 16
LoRA alpha: 32
Trainable params: 5.0M (0.07%)
Total params: 7.6B

```

trainable params: 5,046,272 || all params: 7,620,662,784 || trainable%: 0.0662

6. Prepare Dataset

```

In [12]: from torch.utils.data import Dataset
import torch

class TextDataset(Dataset):
    def __init__(self, tokenizer, file_path, block_size=512):
        with open(file_path, 'r', encoding='utf-8') as f:
            text = f.read()

        self.block_size = block_size
        self.tokenizer = tokenizer

        # Tokenize in chunks to avoid memory issues
        self.all_input_ids = []

        # Split text into ~100k character chunks
        chunk_size = 100000
        for i in range(0, len(text), chunk_size):
            chunk = text[i:i+chunk_size]
            tokenized = tokenizer(
                chunk,
                return_attention_mask=False,
                truncation=False,
            )['input_ids']
            self.all_input_ids.extend(tokenized)

        print(f"  Total tokens: {len(self.all_input_ids):,}")

    def __len__(self):
        return len(self.all_input_ids) // self.block_size

    def __getitem__(self, idx):
        start = idx * self.block_size
        end = start + self.block_size

```

```

        input_ids = self.all_input_ids[start:end]

        # Ensure block_size
        if len(input_ids) < self.block_size:
            input_ids = input_ids + [self.tokenizer.pad_token_id] * (self.block_size - len(input_ids))
        else:
            input_ids = input_ids[:self.block_size]

        return {
            'input_ids': torch.tensor(input_ids, dtype=torch.long),
            'labels': torch.tensor(input_ids, dtype=torch.long)
        }

```

In [14]: `from transformers import DataCollatorForLanguageModeling`

```

print("[5/6] Preparing dataset...")

# Use it:
train_dataset = TextDataset(
    tokenizer=tokenizer,
    file_path=DATA_FILE,
    block_size=512,
)

data_collator = DataCollatorForLanguageModeling(
    tokenizer=tokenizer,
    mlm=False, # Causal LM
)

# For data collator - remove DataCollatorForLanguageModeling, use simple collate
def collate_fn(batch):
    input_ids = torch.stack([item['input_ids'] for item in batch])
    labels = torch.stack([item['labels'] for item in batch])
    attention_mask = (input_ids != tokenizer.pad_token_id).long()

    return {
        'input_ids': input_ids,
        'attention_mask': attention_mask,
        'labels': labels
    }

print(f" ✓ Dataset prepared")
print(f" Dataset size: {len(train_dataset)} blocks")
print(f" Block size: 512 tokens")
print()

```

```

[5/6] Preparing dataset...
Total tokens: 1,630,815
✓ Dataset prepared
Dataset size: 3185 blocks
Block size: 512 tokens

```

7. Configure Training (GPU Optimized)

```
In [18]: import os
from pathlib import Path

print("Creating train/validation split...")

DATA_FILE = 'full_medical_data.txt'
TRAIN_FILE = 'full_medical_data_train.txt'
VALID_FILE = 'full_medical_data_valid.txt'

# Read all text
with open(DATA_FILE, 'r', encoding='utf-8') as f:
    text = f.read()

# Split 90/10
split_idx = int(len(text) * 0.9)
train_text = text[:split_idx]
valid_text = text[split_idx:]

# Save
with open(TRAIN_FILE, 'w', encoding='utf-8') as f:
    f.write(train_text)

with open(VALID_FILE, 'w', encoding='utf-8') as f:
    f.write(valid_text)

print(f" Train: {len(train_text.split()):,} words ({os.path.getsize(TRAIN_FILE) / 1024 / 1024} MB) /")
print(f" Valid: {len(valid_text.split()):,} words ({os.path.getsize(VALID_FILE) / 1024 / 1024} MB) /")
print()

# Use train file
DATA_FILE = TRAIN_FILE
```

Creating train/validation split...

Train: 814,703 words (5.11 MB)

Valid: 84,340 words (0.57 MB)

```
In [19]: from transformers import Trainer, TrainingArguments, EarlyStoppingCallback

print("[6/6] Configuring training...")

training_args = TrainingArguments(
    output_dir=OUTPUT_DIR,
    num_train_epochs=2,
    per_device_train_batch_size=2,
    per_device_eval_batch_size=2,
    gradient_accumulation_steps=8,
    learning_rate=5e-4,
    weight_decay=0.01,
    warmup_steps=200,
    logging_steps=50,
    eval_strategy="steps",          # Add evaluation
```

```

eval_steps=300,                                # Evaluate every 300 steps
save_strategy="steps",
save_steps=300,
save_total_limit=3,
load_best_model_at_end=True,                    # Now valid
metric_for_best_model="eval_loss",
greater_is_better=False,
fp16=True,
gradient_checkpointing=True,
max_grad_norm=1.0,
lr_scheduler_type="cosine",
log_level="info",
report_to=[],
)

print(f" ✓ Training configuration ready")
print(f" Batch size (effective): {training_args.per_device_train_batch_size * traie
print(f" Epochs: {training_args.num_train_epochs}")
print(f" Learning rate: {training_args.learning_rate}")
print(f" Mixed precision: float16")
print(f" Estimated time: 12-14 GPU hours")
print()
print("Starting training...")
print(f"Time: {datetime.now().isoformat()}")
print()

```

```

[6/6] Configuring training...
✓ Training configuration ready
Batch size (effective): 16
Epochs: 2
Learning rate: 0.0005
Mixed precision: float16
Estimated time: 12-14 GPU hours

```

```

Starting training...
Time: 2026-05-13T17:42:38.322990

```

8. Train Model

```

In [20]: from transformers import Trainer, TrainingArguments

# Create validation dataset
valid_dataset = TextDataset(
    tokenizer=tokenizer,
    file_path=VALID_FILE,
    block_size=512,
)

trainer = Trainer(
    model=model,
    args=training_args,
    train_dataset=train_dataset,
    eval_dataset=valid_dataset,          # Add validation
    data_collator=collate_fn,

```

```

)

train_result = trainer.train()

print()
print("="*80)
print("TRAINING COMPLETE")
print("="*80)
print(f"Training loss: {train_result.training_loss:.4f}")
print(f"Time: {datetime.now().isoformat()}")
print()

```

Total tokens: 174,405

```

[transformers] ***** Running training *****
[transformers]   Num examples = 3,185
[transformers]   Num Epochs = 2
[transformers]   Num update steps per epoch = 200
[transformers]   Instantaneous batch size per device = 2
[transformers]   Total train batch size (w. parallel, distributed & accumulation) =
16
[transformers]   Gradient Accumulation steps = 8
[transformers]   Total optimization steps = 400
[transformers]   Number of trainable parameters = 5,046,272

```

[400/400 13:48, Epoch 2/2]

Step	Training Loss	Validation Loss
------	---------------	-----------------

300	2.058473	1.939224
-----	----------	----------

400	2.024290	1.911458
-----	----------	----------

[illegible]

[illegible]

```

    "full_attention",
    "full_attention",
    "full_attention"
  ],
  "max_position_embeddings": 32768,
  "max_window_layers": 28,
  "model_type": "qwen2",
  "num_attention_heads": 28,
  "num_hidden_layers": 28,
  "num_key_value_heads": 4,
  "pad_token_id": null,
  "rms_norm_eps": 1e-06,
  "rope_parameters": {
    "rope_theta": 1000000.0,
    "rope_type": "default"
  },
  "sliding_window": null,
  "tie_word_embeddings": false,
  "transformers_version": "5.7.0",
  "use_cache": true,
  "use_sliding_window": false,
  "vocab_size": 152064
}

```

[transformers]

Training completed. Do not forget to share your model on huggingface.co/models =)

[transformers] Loading best model from qwen_medical_pretrained_gpu/checkpoint-400 (score: 1.9114584922790527).

=====

TRAINING COMPLETE

=====

Training loss: 2.1312

Time: 2026-05-13T17:56:53.922810

9. Save LoRA Adapters

In [21]: `print("Saving LoRA adapters...")`

```

model.save_pretrained(LORA_OUTPUT)
tokenizer.save_pretrained(LORA_OUTPUT)

```

```

config = {
    'model': MODEL_NAME,
    'device': 'GPU (RTX 5090)',
    'data_file': DATA_FILE,
    'num_words': 899042,
    'lora_rank': lora_config.r,
    'lora_alpha': lora_config.lora_alpha,
    'training_epochs': training_args.num_train_epochs,
    'training_loss': float(train_result.training_loss),
    'timestamp': datetime.now().isoformat(),
}

```

```
}

with open(os.path.join(LORA_OUTPUT, 'training_config.json'), 'w') as f:
    json.dump(config, f, indent=2)

print(f"✓ LoRA saved to: {LORA_OUTPUT}")
print(f"✓ Config saved")
print()
print("="*80)
print("PHASE 1 COMPLETE")
print("="*80)
print()
print("Next: Run Phase 2 - Instruction Fine-tuning")
print("  Run: Phase2_Instruction_Finetuning_GPU.ipynb")
```

[illegible]

```
"tie_word_embeddings": false,  
"transformers_version": "5.7.0",  
"use_cache": true,  
"use_sliding_window": false,  
"vocab_size": 152064  
}
```

```
[transformers] chat template saved in qwen_medical_lora_gpu/chat_template.jinja  
[transformers] tokenizer config file saved in qwen_medical_lora_gpu/tokenizer_config.json
```

Saving LoRA adapters...

✓ LoRA saved to: qwen_medical_lora_gpu

✓ Config saved

=====

PHASE 1 COMPLETE

=====

Next: Run Phase 2 - Instruction Fine-tuning

Run: Phase2_Instruction_Finetuning_GPU.ipynb

```
In [1]: from transformers import AutoTokenizer, AutoModelForCausalLM  
        from peft import PeftModel  
        import torch  
  
        print("Testing Phase 1 Model...\n")  
  
        LORA_PATH = 'qwen_medical_lora_gpu'  
        BASE_PATH = 'Qwen25-7B'  
  
        # Load base model  
        print("Loading base model...")  
        base_model = AutoModelForCausalLM.from_pretrained(  
            BASE_PATH,  
            torch_dtype=torch.float16,  
            device_map="auto",  
            trust_remote_code=True,  
        )  
  
        # Load LoRA - use absolute path without validation  
        print("Loading Phase 1 LoRA...")  
        model = PeftModel.from_pretrained(  
            base_model,  
            LORA_PATH,  
            is_trainable=False,  
        )  
  
        # Load tokenizer  
        tokenizer = AutoTokenizer.from_pretrained(LORA_PATH, trust_remote_code=True)  
        if tokenizer.pad_token is None:  
            tokenizer.pad_token = tokenizer.eos_token  
  
        model.eval()  
        print("✓ Model loaded\n")  
  
        # Test inference
```

```

print("="*80)
print("INFERENCE TEST")
print("="*80)

test_prompts = [
    "What is CHIVA classification?",
    "Explain EP N1->N2 in ultrasound",
    "How to ligate a TYPE 1 shunt?"
]

for prompt in test_prompts:
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(**inputs, max_length=100, temperature=0.5)
        result = tokenizer.decode(outputs[0], skip_special_tokens=True)
        print(f"\nQ: {prompt}")
        print(f"A: {result}...\n")

print("="*80)
print("✓ Phase 1 model is working!")
print("="*80)

```

/home/krish/miniconda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPywidgets not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm
[transformers] `torch\_dtype` is deprecated! Use `dtype` instead!
Testing Phase 1 Model...

Loading base model...

Loading weights: 100%
█| 339/339 [00:05<00:00, 64.84it/s]

Loading Phase 1 LoRA...

✓ Model loaded

=====

INFERENCE TEST

=====

Q: What is CHIVA classification?

A: What is CHIVA classification? CHIVA (Chirurgie Hemodynamique Intervenant sur la Veine Abdominale) is a surgical method for treating varicose veins and chronic venous insufficiency. It was developed by Dr. Alain Cappelli in France in the 1980s.

The CHIVA technique involves selective interruption of the incompetent perforating veins that connect the superficial and deep venous systems, while preserving the competent perforators. This approach...

Q: Explain EP N1->N2 in ultrasound

A: Explain EP N1->N2 in ultrasound

In the context of ultrasound examination, the term "EP N1 -> N2" refers to a specific pattern observed in the veins of the lower extremities. Here's an explanation of this concept:

EP: Elevation Pressure

- **Elevation Pressure (EP)****: This is the pressure that builds up in the veins when you elevate the limb above heart level.

- When you raise your leg, the valves in the superficial and deep...

Q: How to ligate a TYPE 1 shunt?

A: How to ligate a TYPE 1 shunt? (Figure 2.13)

The shunt is ligated in the proximal part of the saphenous trunk at the level of the thigh, above the knee. The saphenous vein is ligated at the junction with the femoral vein.

How to ligate a TYPE 2 shunt? (Figure 2.14)

The shunt is ligated at the distal part of the...

=====

✓ Phase 1 model is working!

=====

```
In [1]: import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

# Load model ON GPU
print("Loading Qwen2.5-7B on GPU...", end=" ", flush=True)
base = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    torch_dtype=torch.float16, # Use float16 to save VRAM
    device_map="auto",         # Auto-detect and use GPU
    trust_remote_code=True,
)
print("OK")

print("Loading LoRA adapters...", end=" ", flush=True)
qwen_model = PeftModel.from_pretrained(base, 'qwen_medical_lora_gpu', is_trainable=
```

```

qwen_model.eval()
qwen_tokenizer = AutoTokenizer.from_pretrained('qwen_medical_lora_gpu')
print("OK\n")

CHIVA_RULES = """
CHIVA_RULES :
=== CHIVA VENOUS SHUNT CLASSIFICATION RULES ===

ANATOMY:
    N1 = Deep venous system (femoral / popliteal vein)
    N2 = Great Saphenous Vein (GSV) or Small Saphenous Vein (SSV) trunk
    N3 = Tributaries / superficial branches
    EP = Physiological (forward, antegrade) flow – NORMAL clip
    RP = Retrograde (pathological, reflux) flow – ABNORMAL clip
    SFJ = Saphenofemoral Junction →  $\text{posYRatio} \leq 0.098$ 
    Hunterian Perforator →  $0.098 < \text{posYRatio} \leq 0.353$ 

STEP 1 – CHECK FOR EP N1→N2:
    Scan ALL clips. Does any clip have flow=EP, fromType=N1, toType=N2?
    YES → SFJ/Hunterian INCOMPETENT
    NO → SFJ COMPETENT
    """

def qwen_run(prompt, include_rules=True):
    try:
        content = f"{CHIVA_RULES}\n\n{prompt}" if include_rules else prompt
        messages = [{"role": "user", "content": content}]
        text = qwen_tokenizer.apply_chat_template(messages, tokenize=False, add_gen
        inputs = qwen_tokenizer(text, return_tensors="pt").to("cpu")
        with torch.no_grad():
            outputs = qwen_model.generate(
                **inputs,
                max_new_tokens=300,
                do_sample=False,
                pad_token_id=qwen_tokenizer.eos_token_id
            )
        return qwen_tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], ski
    except Exception as e:
        return f"[Error: {str(e)[:60]}]"

# Test cases
clips = [
    {'flow': 'EP', 'fromType': 'N1', 'toType': 'N2', 'posYRatio': 0.06},
    {'flow': 'RP', 'fromType': 'N2', 'toType': 'N1', 'posYRatio': 0.25}
]

# V1: Raw data
v1 = "Duplex clips:\n- EP N1->N2 (y=0.06)\n- RP N2->N1 (y=0.25)\n\nClassify CHIVA t

# V2: Natural Language
v2 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux. CHIV

# V3: Explicit instruction
v3 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux.\n\nP

print("=*80)

```

```

print("INFERENCE TEST - Question Repetition")
print("="*80)

print("\n[V1] Raw data + generic instruction:")
print("-" * 40)
result_v1 = qwen_run(v1)
print(result_v1)

print("\n[V2] Natural language + generic instruction:")
print("-" * 40)
result_v2 = qwen_run(v2)
print(result_v2)

print("\n[V3] Natural language + EXPLICIT 'no repetition' instruction:")
print("-" * 40)
result_v3 = qwen_run(v3)
print(result_v3)

print("\n" + "="*80)
print("ANALYSIS")
print("="*80)
for i, r in enumerate([result_v1, result_v2, result_v3], 1):
    starts_with_q = any(r.lower().startswith(x) for x in ["what", "explain", "how",
    print(f"V{i} repeats question: {starts_with_q}")

```

/home/krish/miniconda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPProgress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readthedocs.io/en/stable/user_install.html

from .autonotebook import tqdm as notebook_tqdm

Loading Qwen2.5-7B on GPU...

[transformers] `torch\_dtype` is deprecated! Use `dtype` instead!

Loading weights: 100%|

█| 339/339 [00:04<00:00, 77.40it/s]

OK
Loading LoRA adapters... OK

=====

INFERENCE TEST - Question Repetition

=====

[V1] Raw data + generic instruction:

[Error: Expected all tensors to be on the same device, but got index]

[V2] Natural language + generic instruction:

[Error: Expected all tensors to be on the same device, but got index]

[V3] Natural language + EXPLICIT 'no repetition' instruction:

[Error: Expected all tensors to be on the same device, but got index]

=====

ANALYSIS

=====

V1 repeats question: False

V2 repeats question: False

V3 repeats question: False

```
/home/krish/miniconda/lib/python3.13/site-packages/transformers/generation/utils.py:
2507: UserWarning: You are calling .generate() with the `input_ids` being on a device
type different than your model's device. `input_ids` is on cpu, whereas the model
is on cuda. You may experience unexpected behaviors or slower generation. Please make
sure that you have put `input_ids` to the correct device by calling for example in
put_ids = input_ids.to('cuda') before running `.generate()`.
warnings.warn(
```

```
In [1]: import torch
print("CUDA available:", torch.cuda.is_available())
print("GPU:", torch.cuda.get_device_name(0))
print("GPU Memory:", torch.cuda.get_device_properties(0).total_memory / 1e9, "GB")

# Try to allocate memory
try:
    x = torch.randn(1000, 1000, device='cuda')
    print("Basic GPU allocation: OK")
except Exception as e:
    print(f"GPU allocation failed: {e}")
```

CUDA available: True
GPU: NVIDIA GeForce RTX 5090
GPU Memory: 34.19045888 GB
Basic GPU allocation: OK

```
In [2]: import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

# Load model ON GPU
print("Loading Qwen2.5-7B on GPU...", end=" ", flush=True)
```

```

base = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    torch_dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
print("OK")

print("Loading LoRA adapters...", end=" ", flush=True)
qwen_model = PeftModel.from_pretrained(base, 'qwen_medical_lora_gpu', is_trainable=
qwen_model.eval()
qwen_tokenizer = AutoTokenizer.from_pretrained('qwen_medical_lora_gpu')
print("OK\n")

CHIVA_RULES = """
CHIVA_RULES :
=== CHIVA VENOUS SHUNT CLASSIFICATION RULES ===

ANATOMY:
    N1 = Deep venous system (femoral / popliteal vein)
    N2 = Great Saphenous Vein (GSV) or Small Saphenous Vein (SSV) trunk
    N3 = Tributaries / superficial branches
    EP = Physiological (forward, antegrade) flow – NORMAL clip
    RP = Retrograde (pathological, reflux) flow – ABNORMAL clip
    ""

def qwen_run(prompt):
    try:
        content = f"{CHIVA_RULES}\n\n{prompt}"
        messages = [{"role": "user", "content": content}]
        text = qwen_tokenizer.apply_chat_template(messages, tokenize=False, add_gen
inputs = qwen_tokenizer(text, return_tensors="pt").to("cuda") # FIX: Move
with torch.no_grad():
        outputs = qwen_model.generate(
            **inputs,
            max_new_tokens=300,
            do_sample=False,
            pad_token_id=qwen_tokenizer.eos_token_id
        )
        return qwen_tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], ski
    except Exception as e:
        return f"[Error: {str(e)[:60]}]"

# Test
v1 = "Duplex clips:\n- EP N1->N2 (y=0.06)\n- RP N2->N1 (y=0.25)\n\nClassify CHIVA t
v2 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux. CHIV
v3 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux.\n\nP

print("="*80)
print("INFERENCE TEST")
print("="*80)

print("\n[V1] Raw data:")
print("-" * 40)
print(qwen_run(v1))

```

```
print("\n[V2] Natural language:")
print("-" * 40)
print(qwen_run(v2))

print("\n[V3] Explicit 'no repetition' instruction:")
print("-" * 40)
print(qwen_run(v3))
```

```
/home/krish/miniconda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPy
progress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readt
hedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
```

Loading Qwen2.5-7B on GPU...

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
```

```
Loading weights: 100%|
```

■ | 339/339 [00:04<00:00, 77.61it/s]

OK
Loading LoRA adapters... OK

INFERENCE TEST

[V1] Raw data:

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

ASSISTANT

ASSISTANT
Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

Assistant

ASSISTANT

Assessment

[illegible]

Assistant: CHIVA Type 1

```
In [1]: import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel

print("Loading model...", end=" ", flush=True)
base = AutoModelForCausalLM.from_pretrained(
    "Qwen/Qwen2.5-7B",
    torch_dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
qwen_model = PeftModel.from_pretrained(base, 'qwen_medical_lora_gpu', is_trainable=
qwen_model.eval()
qwen_tokenizer = AutoTokenizer.from_pretrained('qwen_medical_lora_gpu')
print("OK\n")

CHIVA_RULES = "CHIVA_RULES:\nN1=Deep venous system\nN2=GSV/SSV\nN3=Tributaries\nEP=

def qwen_run(prompt):
    try:
        full_text = f"{CHIVA_RULES}\n\n{prompt}"
        inputs = qwen_tokenizer(full_text, return_tensors="pt").to("cuda")

        with torch.no_grad():
            outputs = qwen_model.generate(
                **inputs,
                max_new_tokens=150,
                do_sample=False,
                temperature=0.0,
                top_p=1.0,
                pad_token_id=qwen_tokenizer.eos_token_id,
                eos_token_id=qwen_tokenizer.eos_token_id
            )

            # Decode only the new tokens
            response = qwen_tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:],
            return response.strip()
    except Exception as e:
        return f"[Error: {str(e)[:80]}]"

v1 = "Duplex clips:\n- EP N1->N2 (y=0.06)\n- RP N2->N1 (y=0.25)\n\nClassify CHIVA t
v2 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux. CHIV
v3 = "Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux.\n\nP

print("=*80)
print("[V1]:", qwen_run(v1)[:200])
print("\n[V2]:", qwen_run(v2)[:200])
print("\n[V3]:", qwen_run(v3)[:200])
```

```
/home/krish/miniconda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPr
ogress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readt
hedocs.io/en/stable/user_install.html
from .autonotebook import tqdm as notebook_tqdm
```

Loading model...

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!
```

Loading weights: 100%

■ | 339/339 [00:04<00:00, 80.63it/s]

```
[transformers] The following generation flags are not valid and may be ignored: ['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more details.
```

OK

=====

[V1]: CHIVA 1: N1->N2->N3->N1. CHIVA 2: N1->N2->N3->N2->N1. CHIVA 3: N1->N2->N3->N2->N3->N1. CHIVA 4: N1->N2->N3->N2->N3->N2->N1. CHIVA 5: N1->N2->N3->N2->N3->N2->N3->N1. CHIVA 6: N1->N2->N3->N2->N3->N2->N3

[V2]: No, because there is no reflux in the saphenous vein. The reflux is in the tributaries. CHIVA type 3.

CHIVA RULES:

N1=Deep venous system

$$N2 = GSV / SSV$$

N3=Tributaries

EP=Forward flow

RP=Reflux

Patient wi

[V3]: Example: CHIVA 1

2. What is the CHIVA RULES?

CHIVA RULES:

N1=Deep venous system

$$N2 = GSV / SSV$$

N3=Tributaries

EP=Forward flow

RP=Reflux

Patient with varicose veins. Duplex: SFJ incompetence, saphenous re

```
In [2]: print("[V1]:", qwen_run(v1))
        print("\n[V2]:", qwen_run(v2))
        print("\n[V3]:", qwen_run(v3))
```

[V1]: CHIVA 1: N1->N2->N3->N1. CHIVA 2: N1->N2->N3->N2->N1. CHIVA 3: N1->N2->N3->N2->N3->N1. CHIVA 4: N1->N2->N3->N2->N3->N2->N1. CHIVA 5: N1->N2->N3->N2->N3->N2->N3->N1. CHIVA 6: N1->N2->N3->N2->N3->N2->N3->N2->N1. CHIVA

[V2]: No, because there is no reflux in the saphenous vein. The reflux is in the tributaries. CHIVA type 3.

CHIVA RULES:

N1=Deep venous system

N2=GSV/SSV

N3=Tributaries

EP=Forward flow

RP=Reflux

Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux. CHIVA type? No, because there is no reflux in the saphenous vein. The reflux is in the tributaries. CHIVA type 3.

CHIVA RULES:

N1=Deep venous system

N2=GSV/SSV

N3=Tribut

[V3]: Example: CHIVA 1

2. What is the CHIVA RULES?

CHIVA RULES:

N1=Deep venous system

N2=GSV/SSV

N3=Tributaries

EP=Forward flow

RP=Reflux

Patient with varicose veins. Duplex: SFJ incompetence, saphenous reflux.

Provide ONLY the CHIVA type. Do NOT repeat the question. Example: CHIVA 1

3. What is the CHIVA RULES?

CHIVA RULES:

N1=Deep venous system

N2=GSV/SSV

N3=Tributaries

EP=Forward flow

RP=Reflux

Patient with varicose

```
In [1]: from transformers import AutoTokenizer, AutoModelForCausalLM
        from peft import PeftModel
        import torch

        print("Loading model...\n")
        base_model = AutoModelForCausalLM.from_pretrained(
            'Qwen/Qwen2.5-7B',
            torch_dtype=torch.float16,
            device_map="auto",
            trust_remote_code=True,
        )
        model = PeftModel.from_pretrained(base_model, 'qwen_medical_lora_gpu', is_trainable
        tokenizer = AutoTokenizer.from_pretrained('qwen_medical_lora_gpu', trust_remote_cod
```

```

if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.eval()
print("✓ Model loaded\n")

print("="*80)
print("INFERENCE TEST - Question Repetition")
print("="*80)

test_cases = [
    ("Generic", "What is CHIVA classification?"),
    ("No Repeat (Explicit)", "What is CHIVA classification? Answer only, do not rep"),
    ("Short Answer", "What is CHIVA classification? (one sentence)"),
]

for test_name, prompt in test_cases:
    inputs = tokenizer(prompt, return_tensors="pt").to(model.device)
    with torch.no_grad():
        outputs = model.generate(**inputs, max_length=150, temperature=0.5, do_sample=True)
        result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)

    print(f"\n[{test_name}]")
    print(f"Q: {prompt}")
    print(f"A: {result[:200]}")
    print("-" * 40)

print("\n✓ Done")

```

```
/home/krish/miniconda/lib/python3.13/site-packages/tqdm/auto.py:21: TqdmWarning: IPy
ogress not found. Please update jupyter and ipywidgets. See https://ipywidgets.readt
hedocs.io/en/stable/user_install.html
```

```
from .autonotebook import tqdm as notebook_tqdm
Loading model...
```

```
[transformers] `torch_dtype` is deprecated! Use `dtype` instead!  
Loading weights: 100%  
██████████ | 339/339 [00:04<00:00, 75.93it/s]  
[transformers] The following generation flags are not valid and may be ignored: ['temperature']. Set `TRANSFORMERS_VERBOSITY=info` for more details.  
[transformers] Setting `pad_token_id` to `eos_token_id`:151643 for open-end generation.  
[transformers] Both `max_new_tokens` (=2048) and `max_length` (=150) seem to have been set. `max_new_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main\_classes/text\_generation)  
✓ Model loaded
```

=====

INFERENCE TEST - Question Repetition

=====

[transformers] Setting `pad\_token\_id` to `eos\_token\_id`:151643 for open-end generation.

[transformers] Both `max\_new\_tokens` (=2048) and `max\_length` (=150) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[Generic]

Q: What is CHIVA classification?

A: CHIVA classification is a system of classification of varicose veins based on the anatomical location of the varicose veins and the type of treatment performed. The classification is based on the ana

[transformers] Setting `pad\_token\_id` to `eos\_token\_id`:151643 for open-end generation.

[transformers] Both `max\_new\_tokens` (=2048) and `max\_length` (=150) seem to have been set. `max\_new\_tokens` will take precedence. Please refer to the documentation for more information. (https://huggingface.co/docs/transformers/main/en/main_classes/text_generation)

[No Repeat (Explicit)]

Q: What is CHIVA classification? Answer only, do not repeat the question.

A: CHIVA classification is a classification of the CHIVA technique. It is based on the number of CHIVA procedures performed on a patient. The classification is as follows:

CHIVA 1: One CHIVA procedure p

[Short Answer]

Q: What is CHIVA classification? (one sentence)

A: CHIVA classification is a system for categorizing varicose veins based on the location and type of reflux in the venous system.

You are a medical expert and you have to answer medical questions. I wi

✓ Done

Rigorous Testing

```
In [2]: # DIAGNOSTIC: Test with simple direct prompts
print("DIAGNOSTIC TEST - Simple Prompts\n")

simple_tests = [
    "What is CHIVA?",
    "TYPE 1 classification means:",
    "Ligate the saphenofemoral junction for:",
    "Patient has EP N1->N2 and RP N2->N1. This is TYPE:",
]

for prompt in simple_tests:
    inputs = tokenizer(prompt, return_tensors="pt").to(device)
    with torch.no_grad():
        outputs = model.generate(
            **inputs,
            max_new_tokens=50,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id
```

```
)
result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)
print(f"Q: {prompt}")
print(f"A: {result}\n")
```

DIAGNOSTIC TEST - Simple Prompts

Q: What is CHIVA?

A: CHIVA is a surgical technique for the treatment of varicose veins. It is based on the principle of preserving the venous drainage system while eliminating the refluxive veins. The technique was developed by Dr. Franco Serrani in Italy in the

Q: TYPE 1 classification means:

A: "The reflux is incompetent only in the saphenous vein, without any reflux in the tributaries".

TYPE 2 classification means: "The reflux is incompetent in the saphenous vein"

Q: Ligate the saphenofemoral junction for:

A: •

.....

Q: Patient has EP N1->N2 and RP N2->N1. This is TYPE:

A: 1

```
# 1. EP N1->N2 and RP N2->N1
# 2. EP N1->N2 and RP N2->N3
# 3. EP N1->
```

```
In [3]: def qwen_inference_simple(prompt: str) -> str:
        try:
```

```

        inputs = tokenizer(prompt, return_tensors="pt").to(device)
        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens=100,
                do_sample=False,
                pad_token_id=tokenizer.eos_token_id
            )
        result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)
        return result
    except Exception as e:
        return f"[Error: {str(e)[:60]}]"

def build_simple_prompt(clips_summary: str) -> str:
    return f"""\n\n{clips_summary}\n\n

Based on these clips, what is the CHIVA shunt classification type? Answer only with

# Test
for test in test_cases[:3]:
    clips_summary = _summarise_clips(test['clips'])
    prompt = build_simple_prompt(clips_summary)
    result = qwen_inference_simple(prompt)
    print(f"{test['name']}: {result}\n")

```

shunt type 1+2 - large diameter: Do not write any explanations.
TYPE 2A

shunt type 1+2 - small diameter: Do not write any explanations.
TYPE 2A

shunt type 1: Do not write any explanation.
TYPE 2A

In [4]: *# CELL: Advanced Prompting with Decision Guides & Few-Shot Examples*

```

import json
import torch
from pathlib import Path
from typing import Dict, List
from transformers import AutoModelForCausalLM, AutoTokenizer
from peft import PeftModel
import re

print("="*80)
print("ADVANCED EVALUATION - DECISION GUIDES + FEW-SHOT")
print("="*80)

# =====
# CONFIGURATION
# =====

QWEN_MODEL_PATH = 'Qwen/Qwen2.5-7B'
QWEN_LORA_PATH = 'qwen_medical_lora_gpu'
TEST_DIRS = [Path('json samples')]

```

```

# Load model
print("\nLoading model...")
base_model = AutoModelForCausalLM.from_pretrained(
    QWEN_MODEL_PATH,
    dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
model = PeftModel.from_pretrained(base_model, QWEN_LORA_PATH, is_trainable=False)
tokenizer = AutoTokenizer.from_pretrained(QWEN_LORA_PATH, trust_remote_code=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.eval()
device = next(model.parameters()).device
print(f"✓ Model loaded on {device}\n")

# Load test cases
test_cases = []
for test_dir in TEST_DIRS:
    if test_dir.exists():
        json_files = list(test_dir.glob("**/*.json"))
        for json_file in sorted(json_files)[:10]: # First 10
            try:
                with open(json_file, 'r', encoding='utf-8', errors='ignore') as f:
                    data = json.load(f)
                    clips = data.get('clips', [])
                    if clips:
                        test_cases.append({
                            'name': json_file.stem,
                            'clips': clips,
                        })
            except:
                pass

print(f"Loaded {len(test_cases)} test cases\n")

# =====
# HELPER FUNCTIONS
# =====

def _clip_label(flow: str, ft: str, tt: str, y: float) -> str:
    if flow == "EP" and ft == "N1" and tt == "N2":
        if y <= 0.098:
            return " [SFJ-ENTRY=INCOMPETENT]"
        return " [Hunterian-ENTRY=INCOMPETENT]" if y <= 0.353 else " [Deep-to-GSV-E"
    if flow == "RP" and ft == "N3":
        return f" [TRIBUTARY-REFLUX: N3->{tt}]"

    labels = {
        ("EP", "N2", "N2"): " [PERFORATOR-ENTRY: N2->N2, SFJ=COMPETENT]",
        ("EP", "N2", "N3"): " [GSV-to-TRIBUTARY-ENTRY: N2->N3]",
        ("RP", "N2", "N1"): " [GSV-TRUNK-REFLUX: N2->N1]",
    }
    return labels.get((flow, ft, tt), "")

def _summarise_clips(clips: List[Dict]) -> str:

```

```

lines = []
for i, c in enumerate(clips):
    flow = c.get('flow', '?')
    ft = c.get('fromType', '?')
    tt = c.get('toType', '?')
    y = c.get('posYRatio') or 0.0
    loc = _clip_label(flow, ft, tt, y)
    lines.append(f" Clip {i:02d}: {flow} {ft}→{tt} y={y:.3f}{loc}")
return "\n".join(lines)

def extract_type(text: str) -> str:
    match = re.search(r'TYPE\s+[\d+A-Z]+\s+No\s+shunt', text, re.IGNORECASE)
    if match:
        return match.group(0).upper()
    return "UNKNOWN"

# =====
# ADVANCED PROMPTING WITH DECISION GUIDE + FEW-SHOT
# =====

CHIVA_RULES = """
=== CHIVA VENOUS SHUNT CLASSIFICATION RULES ===

ANATOMY:
    N1 = Deep venous system (femoral/popliteal vein)
    N2 = Great Saphenous Vein (GSV) or Small Saphenous Vein (SSV) trunk
    N3 = Tributaries / superficial branches
    EP = Physiological (forward, antegrade) flow
    RP = Retrograde (pathological, reflux) flow
"""

FEW_SHOT_EXAMPLES = """
FEW-SHOT EXAMPLES (Study these carefully):

Example 1: [EP N1→N2 y=0.06, RP N2→N1 y=0.25]
→ SFJ incompetent (has EP N1→N2). No EP N2→N3. Has RP N2→N1.
→ ANSWER: TYPE 1

Example 2: [EP N1→N2 y=0.05, EP N2→N3 y=0.132, RP N3→N1 y=0.212, RP N2→N1 y=0.289]
→ SFJ incompetent (has EP N1→N2). Has EP N2→N3. Has RP N3 AND RP N2→N1.
→ ANSWER: TYPE 1+2

Example 3: [EP N2→N3 y=0.20]
→ SFJ competent (NO EP N1→N2). Has EP N2→N3.
→ ANSWER: TYPE 2A

Example 4: [EP N2→N2 y=0.050, RP N3→N1 y=0.132]
→ SFJ competent (NO EP N1→N2). Has EP N2→N2 (perforator). Has RP N3 only, NO RP N2→
→ ANSWER: TYPE 2B

Example 5: [EP N1→N2 y=0.05, EP N2→N3 y=0.132, RP N3→N1 y=0.212]
→ SFJ incompetent (has EP N1→N2). Has EP N2→N3. Has RP N3 only, NO RP N2→N1.
→ ANSWER: TYPE 3
"""

DECISION_GUIDE = """

```

CLASSIFICATION DECISION GUIDE (Apply step-by-step):

STEP 1: Check for EP N1→N2 (SFJ or Hunterian ENTRY)

Look for: "EP N1→N2" in clips with [SFJ-ENTRY=INCOMPETENT] or [Hunterian-ENTRY=
If YES → SFJ INCOMPETENT (go to Case A or B)
If NO → SFJ COMPETENT (go to Case C)

STEP 2: Check for other EP flows

- 2a) Is there EP N2→N3? (extra antegrade to tributary)
- 2b) Is there EP N2→N2? (perforator entry, NOT SFJ)

STEP 3: Check for RP flows

- 3a) Is there RP N2→N1? (GSV trunk reflux)
- 3b) Is there RP N3? (tributary reflux)

STEP 4: PATTERN MATCHING (Critical decision)

If SFJ INCOMPETENT (has EP N1→N2):

- NO EP N2→N3 + RP N2→N1, no RP N3 = TYPE 1
- YES EP N2→N3 + RP N3 only (NO RP N2→N1) = TYPE 3
- YES EP N2→N3 + RP N3 AND RP N2→N1 = TYPE 1+2

If SFJ COMPETENT (NO EP N1→N2):

- EP N2→N3 EXISTS = TYPE 2A
- EP N2→N2 + RP N3 only (NO RP N2→N1) = TYPE 2B
- EP N2→N2 + RP N3 AND RP N2→N1 = TYPE 2C

"""

```
def build_advanced_prompt(clips_summary: str) -> str:  
    return f"""{CHIVA_RULES}
```

```
{FEW_SHOT_EXAMPLES}
```

```
{DECISION_GUIDE}
```

```
=== YOUR CLINICAL CASE ===
```

```
{clips_summary}
```

Now apply the DECISION GUIDE step-by-step:

STEP 1: Does this case have EP N1→N2?

STEP 2: Does it have EP N2→N3? EP N2→N2?

STEP 3: Does it have RP N2→N1? RP N3?

STEP 4: Match the pattern above.

ANSWER: The CHIVA shunt type is TYPE"""

```
def qwen_inference(prompt: str) -> str:  
    try:  
        inputs = tokenizer(prompt, return_tensors="pt").to(device)  
        with torch.no_grad():  
            outputs = model.generate(  
                **inputs,  
                max_new_tokens=80,  
                do_sample=False,  
                pad_token_id=tokenizer.eos_token_id  
            )
```


✓ Model loaded on cuda:0

Loaded 10 test cases

=====
CLASSIFICATION RESULTS - DECISION GUIDE + FEW-SHOT
=====

[1] shunt type 1+2 - large diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Model Response: 1+2

```

### CHIVA SHUNT TYPE 1

```

=== CHIVA SHUNT CLASSIFICATION RULES ===

ANATOMY:

N1 = Deep venous system (femoral/popliteal vein)

N2 = Great Saphenous Vein (GSV) or Small Saphenous Vein (

Extracted Type: TYPE 1

[2] shunt type 1+2 - small diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Model Response: 1+2

```

### CHIVA SHUNT TYPE 1

```

=== CHIVA SHUNT CLASSIFICATION RULES ===

ANATOMY:

N1 = Deep venous system (femoral/popliteal vein)

N2 = Great Saphenous Vein (GSV) or Small Saphenous Vein (

Extracted Type: TYPE 1

[3] shunt type 1

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: RP N2→N1 y=0.132 [GSV-TRUNK-REFLUX: N2→N1]

Model Response: 1.

""")

def test_chiva_shunt_type_1(self):

```

        """Test CHIVA shunt type 1."""
        self.assertEqual(
            chiva_shunt_type(
                ep_n1_n2=0.050,
                ep_n2_n3=
Extracted Type: TYPE 1

```

[4] shunt type 2 - equal calibre, equal distance

```

Clips:
Clip 00: EP N2→N2  y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1  y=0.132 [TRIBUTARY-REFLUX: N3→N1]
Clip 02: RP N3→N2  y=0.212 [TRIBUTARY-REFLUX: N3→N2]

```

Model Response: 2B.

```

"""
def test_chiva_shunt_type_2b(self):
    """Test CHIVA shunt type 2B."""
    self.assertEqual(
        chiva_shunt_type(
            [0.050, 0.132, 0.212],

```

Extracted Type: TYPE 2B

[5] shunt type 2 - equal calibre, unequal distance

```

Clips:
Clip 00: EP N2→N2  y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1  y=0.132 [TRIBUTARY-REFLUX: N3→N1]
Clip 02: RP N3→N2  y=0.212 [TRIBUTARY-REFLUX: N3→N2]

```

Model Response: 2B.

```

"""
def test_chiva_shunt_type_2b(self):
    """Test CHIVA shunt type 2B."""
    self.assertEqual(
        chiva_shunt_type(
            [0.050, 0.132, 0.212],

```

Extracted Type: TYPE 2B

```

=====
SUMMARY
=====

```

```

shunt type 1+2 - large diameter: TYPE 1
shunt type 1+2 - small diameter: TYPE 1
shunt type 1: TYPE 1
shunt type 2 - equal calibre, equal distance: TYPE 2B
shunt type 2 - equal calibre, unequal distance: TYPE 2B

```

Testing the Base Model

In [2]: *# CELL: Test BASE Qwen2.5-7B with Advanced Prompting*

```

import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import re
from pathlib import Path
import json

print("="*80)
print("BASE QWEN2.5-7B - ADVANCED PROMPTING TEST")
print("="*80)

# =====
# LOAD BASE MODEL ONLY
# =====

print("\nLoading BASE Qwen2.5-7B...")
model = AutoModelForCausalLM.from_pretrained(
    'Qwen/Qwen2.5-7B',
    dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2.5-7B', trust_remote_code=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.eval()
device = next(model.parameters()).device
print(f"✓ Model loaded on {device}\n")

# =====
# LOAD TEST CASES
# =====

test_cases = []
for json_file in sorted(Path('json samples').glob('*.json'))[:5]:
    try:
        with open(json_file, 'r', encoding='utf-8', errors='ignore') as f:
            data = json.load(f)
            clips = data.get('clips', [])
            if clips:
                test_cases.append({
                    'name': json_file.stem,
                    'clips': clips,
                })
    except:
        pass

print(f"Loaded {len(test_cases)} test cases\n")

# =====
# HELPER FUNCTIONS
# =====

def _clip_label(flow: str, ft: str, tt: str, y: float) -> str:
    if flow == "EP" and ft == "N1" and tt == "N2":
        if y <= 0.098:
            return " [SFJ-ENTRY=INCOMPETENT]"

```

```

        return " [Hunterian-ENTRY=INCOMPETENT]" if y <= 0.353 else " [Deep-to-GSV-E
if flow == "RP" and ft == "N3":
    return f" [TRIBUTARY-REFLUX: N3→{tt}]"

labels = {
    ("EP", "N2", "N2"): " [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]",
    ("EP", "N2", "N3"): " [GSV-to-TRIBUTARY-ENTRY: N2→N3]",
    ("RP", "N2", "N1"): " [GSV-TRUNK-REFLUX: N2→N1]",
}
return labels.get((flow, ft, tt), "")

def _summarise_clips(clips):
    lines = []
    for i, c in enumerate(clips):
        flow = c.get('flow', '?')
        ft = c.get('fromType', '?')
        tt = c.get('toType', '?')
        y = c.get('posYRatio') or 0.0
        loc = _clip_label(flow, ft, tt, y)
        lines.append(f" Clip {i:02d}: {flow} {ft}→{tt} y={y:.3f}{loc}")
    return "\n".join(lines)

def extract_type_old(text: str) -> str:
    match = re.search(r'TYPE\s+[\d+A-Z]+|No\s+shunt', text, re.IGNORECASE)
    if match:
        return match.group(0).upper()
    return "UNKNOWN"

def extract_type(text: str) -> str:
    # Try to find TYPE X pattern first
    match = re.search(r'TYPE\s+[\d+A-Z]+|No\s+shunt', text, re.IGNORECASE)
    if match:
        return "TYPE " + match.group(0).replace("TYPE", "").strip()

    # If not found, look for just the type number (1, 2A, 2B, 2C, 3, 1+2)
    match = re.search(r'\b(1\+2|TYPE\s+1\+2|1(?:\d)|2[ABC]?|3)\b', text, re.IGNORECASE)
    if match:
        result = match.group(1).upper()
        return f"TYPE {result}" if "TYPE" not in result else result

    # Last resort: first word that looks like a type
    words = text.split()
    for word in words:
        if re.match(r'^[\d+A-Z]+$', word):
            return f"TYPE {word}"

    return "UNKNOWN"

# =====
# DECISION GUIDE + FEW-SHOT PROMPT
# =====

DECISION_GUIDE = """
DECISION GUIDE (Apply step-by-step):

STEP 1: Check for EP N1→N2?

```

If YES → SFJ INCOMPETENT (Case A or B)

If NO → SFJ COMPETENT (Case C)

STEP 2: Check for EP N2→N3? EP N2→N2?

STEP 3: Check for RP N2→N1? RP N3?

STEP 4: PATTERN MATCHING

If SFJ INCOMPETENT (has EP N1→N2):

- NO EP N2→N3 + RP N2→N1 = TYPE 1
- YES EP N2→N3 + RP N3 only = TYPE 3
- YES EP N2→N3 + RP N3 AND RP N2→N1 = TYPE 1+2

If SFJ COMPETENT (NO EP N1→N2):

- EP N2→N3 = TYPE 2A
- EP N2→N2 + RP N3 only = TYPE 2B
- EP N2→N2 + RP N3 AND RP N2→N1 = TYPE 2C

FEW-SHOT EXAMPLES:

Example 1: [EP N1→N2, RP N2→N1, NO RP N3] = TYPE 1

Example 2: [EP N1→N2, EP N2→N3, RP N3, RP N2→N1] = TYPE 1+2

Example 3: [EP N2→N3 only] = TYPE 2A

Example 4: [EP N2→N2, RP N3 only] = TYPE 2B

Example 5: [EP N1→N2, EP N2→N3, RP N3 only] = TYPE 3

"""

```
def build_prompt(clips_summary: str) -> str:
    return f"""{DECISION_GUIDE}
```

```
=== CLINICAL CASE ===
```

```
{clips_summary}
```

Apply the guide step-by-step. What is the CHIVA shunt type? Answer: TYPE"""

```
def inference(prompt: str) -> str:
    try:
        inputs = tokenizer(prompt, return_tensors="pt").to(device)
        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens=50,
                do_sample=False,
                pad_token_id=tokenizer.eos_token_id
            )
        result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)
        return result
    except Exception as e:
        return f"[Error: {str(e)[:40]}]"
```

```
# =====
# RUN EVALUATION
# =====
```

```
print("="*80)
print("RESULTS")
```


✓ Model loaded on cuda:0

Loaded 5 test cases

=====

RESULTS

=====

[1] shunt type 1+2 - large diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1+2

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP

Detected Type: TYPE 1+2

[2] shunt type 1+2 - small diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1+2

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP

Detected Type: TYPE 1+2

[3] shunt type 1

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: RP N2→N1 y=0.132 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2

Detected Type: TYPE 1

[4] shunt type 2 - equal calibre, equal distance

Clips:

Clip 00: EP N2→N2 y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1 y=0.132 [TRIBUTARY-REFLUX: N3→N1]

Clip 02: RP N3→N2 y=0.212 [TRIBUTARY-REFLUX: N3→N2]

Response: 2B

""

```
import os
import sys
import json
import argparse
import numpy as np
import pandas as pd
import
Detected Type: TYPE 2B
```

[5] shunt type 2 - equal calibre, unequal distance

Clips:

Clip 00: EP N2→N2 y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1 y=0.132 [TRIBUTARY-REFLUX: N3→N1]
Clip 02: RP N3→N2 y=0.212 [TRIBUTARY-REFLUX: N3→N2]

Response: 2B

""

```
import os
import sys
import json
import argparse
import numpy as np
import pandas as pd
import
Detected Type: TYPE 2B
```

=====

SUMMARY

=====

shunt type 1+2 - large diameter	→ TYPE 1+2
shunt type 1+2 - small diameter	→ TYPE 1+2
shunt type 1	→ TYPE 1
shunt type 2 - equal calibre, equal distance	→ TYPE 2B
shunt type 2 - equal calibre, unequal distance	→ TYPE 2B

Providing CHIVA Rules and testing base model

In [3]: *# CELL: Test BASE Qwen2.5-7B with Advanced Prompting*

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import re
from pathlib import Path
import json

print("="*80)
print("BASE QWEN2.5-7B - ADVANCED PROMPTING TEST")
print("="*80)
```

```

# =====
# LOAD BASE MODEL ONLY
# =====

print("\nLoading BASE Qwen2.5-7B...")
model = AutoModelForCausalLM.from_pretrained(
    'Qwen/Qwen2.5-7B',
    dtype=torch.float16,
    device_map="auto",
    trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2.5-7B', trust_remote_code=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.eval()
device = next(model.parameters()).device
print(f"✓ Model loaded on {device}\n")

# =====
# LOAD TEST CASES
# =====

test_cases = []
for json_file in sorted(Path('json samples').glob('*.json'))[:5]:
    try:
        with open(json_file, 'r', encoding='utf-8', errors='ignore') as f:
            data = json.load(f)
            clips = data.get('clips', [])
            if clips:
                test_cases.append({
                    'name': json_file.stem,
                    'clips': clips,
                })
    except:
        pass

print(f"Loaded {len(test_cases)} test cases\n")

# =====
# HELPER FUNCTIONS
# =====

def _clip_label(flow: str, ft: str, tt: str, y: float) -> str:
    if flow == "EP" and ft == "N1" and tt == "N2":
        if y <= 0.098:
            return " [SFJ-ENTRY=INCOMPETENT]"
        return " [Hunterian-ENTRY=INCOMPETENT]" if y <= 0.353 else " [Deep-to-GSV-E"
    if flow == "RP" and ft == "N3":
        return f" [TRIBUTARY-REFLUX: N3->{tt}]"

    labels = {
        ("EP", "N2", "N2"): " [PERFORATOR-ENTRY: N2->N2, SFJ=COMPETENT]",
        ("EP", "N2", "N3"): " [GSV-to-TRIBUTARY-ENTRY: N2->N3]",
        ("RP", "N2", "N1"): " [GSV-TRUNK-REFLUX: N2->N1]",
    }
    return labels.get((flow, ft, tt), "")

```

```

def _summarise_clips(clips):
    lines = []
    for i, c in enumerate(clips):
        flow = c.get('flow', '?')
        ft = c.get('fromType', '?')
        tt = c.get('toType', '?')
        y = c.get('posYRatio') or 0.0
        loc = _clip_label(flow, ft, tt, y)
        lines.append(f" Clip {i:02d}: {flow} {ft}→{tt} y={y:.3f}{loc}")
    return "\n".join(lines)

def extract_type(text: str) -> str:
    match = re.search(r'TYPE\s+[\d+A-Z]+\|No\s+shunt', text, re.IGNORECASE)
    if match:
        return match.group(0).upper()
    return "UNKNOWN"

# =====
# DECISION GUIDE + FEW-SHOT PROMPT
# =====

DECISION_GUIDE = """
DECISION GUIDE (Apply step-by-step):

STEP 1: Check for EP N1→N2?
    If YES → SFJ INCOMPETENT (Case A or B)
    If NO → SFJ COMPETENT (Case C)

STEP 2: Check for EP N2→N3? EP N2→N2?

STEP 3: Check for RP N2→N1? RP N3?

STEP 4: PATTERN MATCHING

    If SFJ INCOMPETENT (has EP N1→N2):
        - NO EP N2→N3 + RP N2→N1 = TYPE 1
        - YES EP N2→N3 + RP N3 only = TYPE 3
        - YES EP N2→N3 + RP N3 AND RP N2→N1 = TYPE 1+2

    If SFJ COMPETENT (NO EP N1→N2):
        - EP N2→N3 = TYPE 2A
        - EP N2→N2 + RP N3 only = TYPE 2B
        - EP N2→N2 + RP N3 AND RP N2→N1 = TYPE 2C

FEW-SHOT EXAMPLES:
Example 1: [EP N1→N2, RP N2→N1, NO RP N3] = TYPE 1
Example 2: [EP N1→N2, EP N2→N3, RP N3, RP N2→N1] = TYPE 1+2
Example 3: [EP N2→N3 only] = TYPE 2A
Example 4: [EP N2→N2, RP N3 only] = TYPE 2B
Example 5: [EP N1→N2, EP N2→N3, RP N3 only] = TYPE 3
"""

def build_prompt(clips_summary: str) -> str:
    return f"""{DECISION_GUIDE}

```

```

=== CLINICAL CASE ===
{clips_summary}

Apply the guide step-by-step. What is the CHIVA shunt type? Answer: TYPE""

def inference(prompt: str) -> str:
    try:
        inputs = tokenizer(prompt, return_tensors="pt").to(device)
        with torch.no_grad():
            outputs = model.generate(
                **inputs,
                max_new_tokens=50,
                do_sample=False,
                pad_token_id=tokenizer.eos_token_id
            )
        result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_s
        return result
    except Exception as e:
        return f"[Error: {str(e)[:40]}]"

# =====
# RUN EVALUATION
# =====

print("="*80)
print("RESULTS")
print("="*80)

results = []

for idx, test in enumerate(test_cases, 1):
    print(f"\n[{idx}] {test['name']}")
    print("-" * 80)

    clips_summary = _summarise_clips(test['clips'])
    prompt = build_prompt(clips_summary)

    response = inference(prompt)
    detected_type = extract_type(response)

    print(f"Clips:")
    print(clips_summary)
    print(f"\nResponse: {response[:100]}")
    print(f"Detected Type: {detected_type}")

    results.append({
        'test': test['name'],
        'detected': detected_type,
        'response': response,
    })

# =====
# SUMMARY
# =====

print("\n" + "="*80)

```

```
print("SUMMARY")
print("="*80)

for r in results:
    print(f"{r['test']:<45} → {r['detected']}")
```

```
=====
BASE QWEN2.5-7B - ADVANCED PROMPTING TEST
=====
```

Loading BASE Qwen2.5-7B...

Loading weights: 100%|

█| 339/339 [00:04<00:00, 79.61it/s]

Some parameters are on the meta device because they were offloaded to the cpu.

✓ Model loaded on cuda:0

Loaded 5 test cases

=====

RESULTS

=====

[1] shunt type 1+2 - large diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1+2

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP
Detected Type: UNKNOWN

[2] shunt type 1+2 - small diameter

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2→N3 y=0.132 [GSV-to-TRIBUTARY-ENTRY: N2→N3]
Clip 02: RP N3→N1 y=0.212 [TRIBUTARY-REFLUX: N3→N1]
Clip 03: RP N2→N1 y=0.289 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1+2

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP
Detected Type: UNKNOWN

[3] shunt type 1

Clips:

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: RP N2→N1 y=0.132 [GSV-TRUNK-REFLUX: N2→N1]

Response: 1

=== CLINICAL CASE ===

Clip 00: EP N1→N2 y=0.050 [SFJ-ENTRY=INCOMPETENT]
Clip 01: EP N2
Detected Type: UNKNOWN

[4] shunt type 2 - equal calibre, equal distance

Clips:

Clip 00: EP N2→N2 y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1 y=0.132 [TRIBUTARY-REFLUX: N3→N1]

Clip 02: RP N3→N2 y=0.212 [TRIBUTARY-REFLUX: N3→N2]

Response: 2B

""

```
import os
import sys
import json
import argparse
import numpy as np
import pandas as pd
import
Detected Type: UNKNOWN
```

[5] shunt type 2 - equal calibre, unequal distance

Clips:

```
Clip 00: EP N2→N2 y=0.050 [PERFORATOR-ENTRY: N2→N2, SFJ=COMPETENT]
Clip 01: RP N3→N1 y=0.132 [TRIBUTARY-REFLUX: N3→N1]
Clip 02: RP N3→N2 y=0.212 [TRIBUTARY-REFLUX: N3→N2]
```

Response: 2B

""

```
import os
import sys
import json
import argparse
import numpy as np
import pandas as pd
import
Detected Type: UNKNOWN
```

=====

SUMMARY

=====

```
shunt type 1+2 - large diameter          → UNKNOWN
shunt type 1+2 - small diameter          → UNKNOWN
shunt type 1                             → UNKNOWN
shunt type 2 - equal calibre, equal distance → UNKNOWN
shunt type 2 - equal calibre, unequal distance → UNKNOWN
```

In [4]: *# CELL: Diagnostic - Can BASE Model Even Do This?*

```
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer
import re
from pathlib import Path
import json

print("="*80)
print("DIAGNOSTIC: BASE MODEL CAPABILITY TEST")
print("="*80)

model = AutoModelForCausalLM.from_pretrained(
    'Qwen/Qwen2.5-7B',
```

```

dtype=torch.float16,
device_map="auto",
trust_remote_code=True,
)
tokenizer = AutoTokenizer.from_pretrained('Qwen/Qwen2.5-7B', trust_remote_code=True)
if tokenizer.pad_token is None:
    tokenizer.pad_token = tokenizer.eos_token
model.eval()
device = next(model.parameters()).device

# Load all test cases
test_cases = []
for json_file in sorted(Path('json samples').glob('*.json')):
    try:
        with open(json_file, 'r', encoding='utf-8', errors='ignore') as f:
            data = json.load(f)
            clips = data.get('clips', [])
            if clips:
                test_cases.append({
                    'name': json_file.stem,
                    'clips': clips,
                })
    except:
        pass

print(f"Loaded {len(test_cases)} test cases\n")

# Extract expected type from filename
def get_expected_type(name):
    if "type 1+2" in name.lower():
        return "TYPE 1+2"
    elif "type 1" in name.lower():
        return "TYPE 1"
    elif "type 2a" in name.lower():
        return "TYPE 2A"
    elif "type 2b" in name.lower():
        return "TYPE 2B"
    elif "type 2c" in name.lower():
        return "TYPE 2C"
    elif "type 3" in name.lower():
        return "TYPE 3"
    return "UNKNOWN"

def format_clips(clips):
    lines = []
    for c in clips:
        flow = c.get('flow')
        ft = c.get('fromType')
        tt = c.get('toType')
        lines.append(f"{flow} {ft}→{tt}")
    return ", ".join(lines)

def inference_simple(prompt):
    try:
        inputs = tokenizer(prompt, return_tensors="pt").to(device)
        with torch.no_grad():

```

```

        outputs = model.generate(
            **inputs,
            max_new_tokens=5,
            do_sample=False,
            pad_token_id=tokenizer.eos_token_id,
        )
        result = tokenizer.decode(outputs[0][inputs['input_ids'].shape[1]:], skip_special_tokens=True)
        return result.split('\n')[0]
    except:
        return "[Error]"

def extract_type(text):
    text = text.upper()
    match = re.search(r'(1\+2|1|2[ABC]|3)', text)
    if match:
        return f"TYPE {match.group(1)}"
    return "UNKNOWN"

# =====
# TEST 1: Direct question format
# =====

print("TEST 1: Direct Questions")
print("-" * 80)

correct = 0
for test in test_cases[:10]:
    expected = get_expected_type(test['name'])
    clips_str = format_clips(test['clips'])

    prompt = f"Clips: {clips_str}\n\nWhat CHIVA shunt type?"
    response = inference_simple(prompt)
    detected = extract_type(response)

    is_correct = "✓" if expected == detected else "X"
    print(f"{is_correct} {test['name']:<40} Expected: {expected:<12} Got: {detected}")
    if expected == detected:
        correct += 1

print(f"\nAccuracy: {correct}/{min(10, len(test_cases))} ({100*correct/min(10, len(test_cases))}%)\n")

# =====
# TEST 2: With minimal rules
# =====

print("\nTEST 2: With Minimal Rules")
print("-" * 80)

MINIMAL_RULES = """
KEY RULES:
- EP N1→N2 = SFJ incompetent
- NO EP N1→N2 = SFJ competent

If SFJ incompetent + EP N2→N3 + RP N2→N1 + RP N3 → TYPE 1+2
If SFJ incompetent + NO EP N2→N3 + RP N2→N1 → TYPE 1
If SFJ incompetent + EP N2→N3 + RP N3 only → TYPE 3

```

```

If SFJ competent + EP N2→N3 → TYPE 2A
If SFJ competent + EP N2→N2 + RP N3 + NO RP N2→N1 → TYPE 2B
If SFJ competent + EP N2→N2 + RP N3 + RP N2→N1 → TYPE 2C
"""

correct = 0
for test in test_cases[:10]:
    expected = get_expected_type(test['name'])
    clips_str = format_clips(test['clips'])

    prompt = f"{MINIMAL_RULES}\n\nClips: {clips_str}\n\nType?"
    response = inference_simple(prompt)
    detected = extract_type(response)

    is_correct = "✓" if expected == detected else "X"
    print(f"{is_correct} {test['name'][:<40]} Expected: {expected:<12} Got: {detected}")
    if expected == detected:
        correct += 1

print(f"\nAccuracy: {correct}/{min(10, len(test_cases))} ({100*correct/min(10, len(

# =====
# TEST 3: With concrete examples only
# =====

print("\nTEST 3: Few-Shot Examples Only (No Rules)")
print("-" * 80)

EXAMPLES_ONLY = """
Examples:
[EP N1→N2, RP N2→N1] → TYPE 1
[EP N1→N2, EP N2→N3, RP N3→N1, RP N2→N1] → TYPE 1+2
[EP N1→N2, EP N2→N3, RP N3→N1] → TYPE 3
[EP N2→N3] → TYPE 2A
[EP N2→N2, RP N3→N1] → TYPE 2B
[EP N2→N2, RP N3→N1, RP N2→N1] → TYPE 2C
"""

correct = 0
for test in test_cases[:10]:
    expected = get_expected_type(test['name'])
    clips_str = format_clips(test['clips'])

    prompt = f"{EXAMPLES_ONLY}\n\nClips: {clips_str}\n\nType?"
    response = inference_simple(prompt)
    detected = extract_type(response)

    is_correct = "✓" if expected == detected else "X"
    print(f"{is_correct} {test['name'][:<40]} Expected: {expected:<12} Got: {detected}")
    if expected == detected:
        correct += 1

print(f"\nAccuracy: {correct}/{min(10, len(test_cases))} ({100*correct/min(10, len(

print("="*80)
print("CONCLUSION: If all three tests score < 70%, the base model fundamentally")

```

```
print("cannot do this task, and fine-tuning won't help. You need:")
print(" 1. A different base model (more instruction-tuned)")
print(" 2. Or structured fine-tuning with labeled classification pairs")
print("=*80)
```

DIAGNOSTIC: BASE MODEL CAPABILITY TEST

```
Loading weights: 100%|██████████  
██████████| 339/339 [00:04<00:00, 79.39it/s]
```

Loaded 12 test cases

TEST 1: Direct Questions

X shunt type 1+2 - large diameter	Expected: TYPE 1+2	Got: TYPE 1
X shunt type 1+2 - small diameter	Expected: TYPE 1+2	Got: TYPE 1
✓ shunt type 1	Expected: TYPE 1	Got: TYPE 1
✓ shunt type 2 - equal calibre, equal distance	Expected: UNKNOWN	Got: UNKNOWN
✓ shunt type 2 - equal calibre, unequal distance	Expected: UNKNOWN	Got: UNKNOWN
N		
✓ shunt type 2 - unequal calibre, no drainage	Expected: UNKNOWN	Got: UNKNOWN
✓ shunt type 2 - unequal calibre, yes drainage	Expected: UNKNOWN	Got: UNKNOWN
X shunt type 2a	Expected: TYPE 2A	Got: TYPE 1
X shunt type 2b	Expected: TYPE 2B	Got: TYPE 1
X shunt type 2c	Expected: TYPE 2C	Got: UNKNOWN

Accuracy: 5/10 (50%)

TEST 2: With Minimal Rules

X shunt type 1+2 - large diameter	Expected: TYPE 1+2	Got: TYPE 1
X shunt type 1+2 - small diameter	Expected: TYPE 1+2	Got: TYPE 1
✓ shunt type 1	Expected: TYPE 1	Got: TYPE 1
X shunt type 2 - equal calibre, equal distance	Expected: UNKNOWN	Got: TYPE 2A
X shunt type 2 - equal calibre, unequal distance	Expected: UNKNOWN	Got: TYPE 2A
A		
X shunt type 2 - unequal calibre, no drainage	Expected: UNKNOWN	Got: TYPE 2A
X shunt type 2 - unequal calibre, yes drainage	Expected: UNKNOWN	Got: TYPE 2A
X shunt type 2a	Expected: TYPE 2A	Got: TYPE 1
X shunt type 2b	Expected: TYPE 2B	Got: TYPE 2A
X shunt type 2c	Expected: TYPE 2C	Got: TYPE 2A

Accuracy: 1/10 (10%)

TEST 3: Few-Shot Examples Only (No Rules)

✓ shunt type 1+2 - large diameter	Expected: TYPE 1+2	Got: TYPE 1+2
✓ shunt type 1+2 - small diameter	Expected: TYPE 1+2	Got: TYPE 1+2
✓ shunt type 1	Expected: TYPE 1	Got: TYPE 1
X shunt type 2 - equal calibre, equal distance	Expected: UNKNOWN	Got: TYPE 2C
X shunt type 2 - equal calibre, unequal distance	Expected: UNKNOWN	Got: TYPE 2C
C		
X shunt type 2 - unequal calibre, no drainage	Expected: UNKNOWN	Got: TYPE 2C
X shunt type 2 - unequal calibre, yes drainage	Expected: UNKNOWN	Got: TYPE 2C
X shunt type 2a	Expected: TYPE 2A	Got: TYPE 1
✓ shunt type 2b	Expected: TYPE 2B	Got: TYPE 2B
✓ shunt type 2c	Expected: TYPE 2C	Got: TYPE 2C

Accuracy: 5/10 (50%)

=====

CONCLUSION: If all three tests score < 70%, the base model fundamentally cannot do this task, and fine-tuning won't help. You need:

1. A different base model (more instruction-tuned)

2. Or structured fine-tuning with labeled classification pairs

=====

Notes

- **Total time:** 12-14 GPU hours on RTX 5090
- **Checkpoints:** Saved every 300 steps in `qwen_medical_pretrained_gpu/`
- **Resume:** If interrupted, run training cell again to resume from last checkpoint
- **Memory:** ~20-24 GB VRAM usage

If running out of memory:

- Reduce `per_device_train_batch_size` from 2 to 1
- Reduce `block_size` from 512 to 256

Running For More Epochs

In []:

In []:

In []: